# Addition is lambda definable

The Open Logic Project contributors

openlogic:lambda-calculus/lambda-definability:local-snapshot-20261013; CC BY 4.0

*Editorial note.* All delivered mathematical text is the author's own, copied from the pinned source file sources/olp-lambda-calculus-lambda-definability/pinned-original-source.tex; no statement or proof step was rewritten, reordered or removed. The counted span is the author environment and its own proof environment. References inside the span are handed to the assembler as navigation stubs; whether each of them is load-bearing is recorded on the stub itself.

*Editorial note.* Card cut from the FREE slice of the lit pool for unit olp-lambda-calculus-lambda-definability; the only cross-reference resolves to a label in the same unit.

*Editorial scope.* Single proposition card cut from the Open Logic Project unit olp-lambda-calculus-lambda-definability (pinned-original-source.tex); statement and proof are the author's own text at source lines 459-462 and 464-490.

*Allowed standard prerequisites (editorial).*

- Successor lambda-definability (lambda-definability unit, Proposition prop:succ-ld, source line 415). cited at proof lines 487-489. The second encoding Add' = lambda a b. a Succ b needs Succ to lambda-define the successor function.
- Church numeral encoding (lambda-definability unit, earlier section: the numeral for n is the term lambda f x. f^n x). used at proof lines 473-478 and 484-485. The numeral for n applied to f and x reduces to f^n x, which drives both reduction chains.

*Source reference.* Original proposition 0.1: sources/olp-lambda-calculus-lambda-definability/pinned-original-source.tex, source lines 415–415. Cited at proof lines 487-489: the successor term Succ lambda-defines the successor function, which the second encoding iterates.

**Proposition 0.1.** *The addition function* Add *is lambda definable.*

*Proof.* Addition is lambda defined by the terms

$$\text{Add} \equiv \lambda ab.\, \lambda fx.\, af(bfx)$$

or, alternatively,

$$\text{Add}' \equiv \lambda ab.\, a\ \text{Succ}\ b.$$

The first addition works as follows: Add first accept two numbers $a$ and $b$. The result is a function that accepts $f$ and $x$ and returns $af(bfx)$. If $a$ and $b$ are Church numerals $\overline{n}$ and $\overline{m}$, this reduces to $f^{n+m}(x)$, which is identical to $f^n(f^m(x))$. Or, slowly:

$$\begin{aligned}(\lambda ab.\,\lambda fx.\,af(bfx))\overline{n}\,\overline{m} &\to \lambda fx.\,\overline{n}\,f(\overline{m}\,fx)\\ &\to \lambda fx.\,\overline{n}\,f(f^m x)\\ &\to \lambda fx.\,f^n(f^m x) \equiv \overline{n+m}.\end{aligned}$$

The second representation of addition Add' works differently: Applied to two Church numerals $\overline{n}$ and $\overline{m}$,

$$\text{Add}'\,\overline{n}\,\overline{m} \to \overline{n}\ \text{Succ}\ \overline{m}.$$

But $\overline{n}fx$ always reduces to $f^n(x)$. So,

$$\overline{n}\ \text{Succ}\ \overline{m} \twoheadrightarrow \text{Succ}^n(\overline{m}).$$

And since Succ lambda defines the successor function, and the successor function applied $n$ times to $m$ gives $n+m$, this in turn reduces to $\overline{n+m}$. □